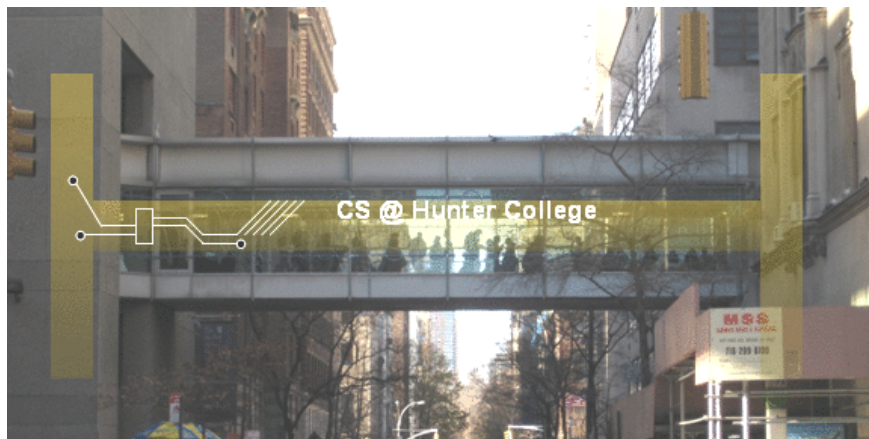


CSci 127: Introduction to Computer Science



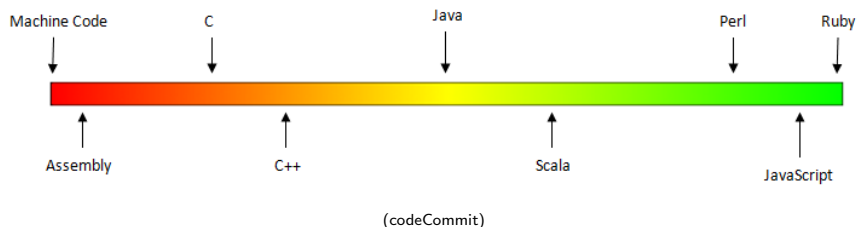
hunter.cuny.edu/csci

Today's Topics



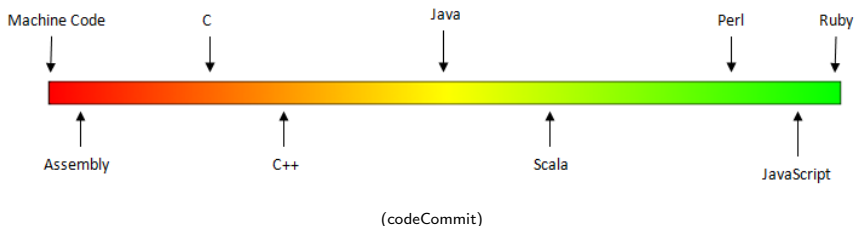
- **Machine Language**
- Loops and Labels in MIPS
- C++: Basic Format & Variables
- I/O and Definite Loops in C++
- Challenge Questions

Low-Level vs. High-Level Languages



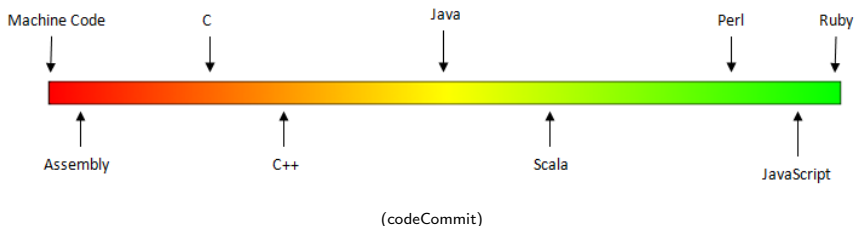
- Can view programming languages on a continuum.

Low-Level vs. High-Level Languages



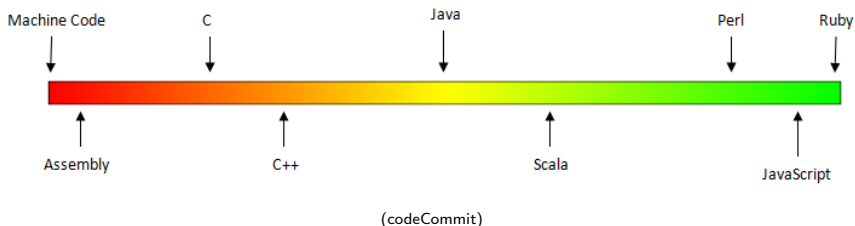
- Can view programming languages on a continuum.
- Those that directly access machine instructions & memory and have little abstraction are **low-level languages**

Low-Level vs. High-Level Languages



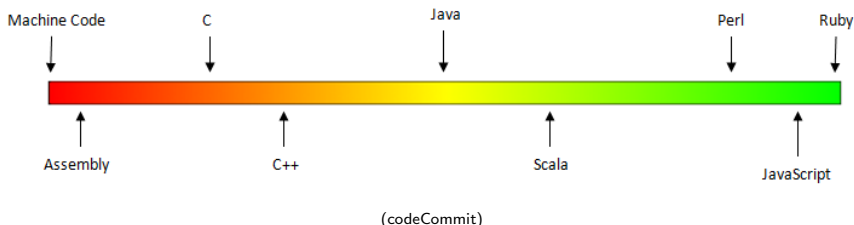
- Can view programming languages on a continuum.
- Those that directly access machine instructions & memory and have little abstraction are **low-level languages** (e.g. machine language, assembly language).

Low-Level vs. High-Level Languages



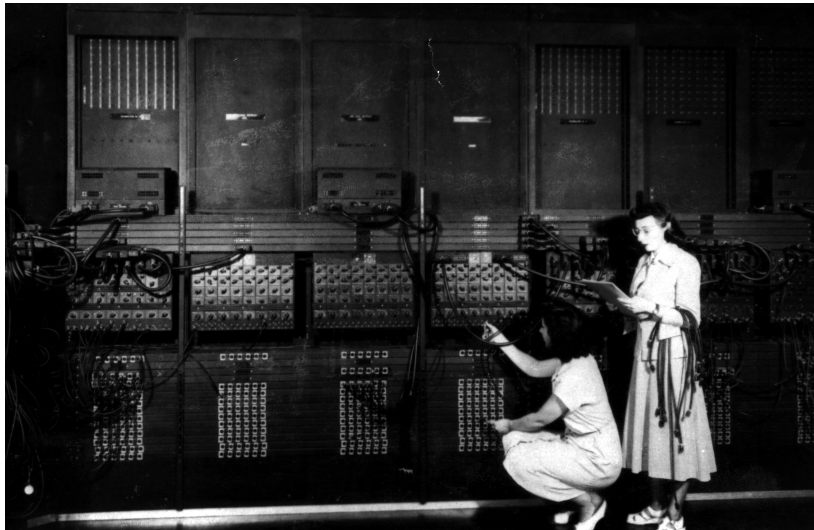
- Can view programming languages on a continuum.
- Those that directly access machine instructions & memory and have little abstraction are **low-level languages** (e.g. machine language, assembly language).
- Those that have strong abstraction (allow programming paradigms independent of the machine details, such as complex variables, functions and looping that do not translate directly into machine code) are called **high-level languages**.

Low-Level vs. High-Level Languages



- Can view programming languages on a continuum.
- Those that directly access machine instructions & memory and have little abstraction are **low-level languages** (e.g. machine language, assembly language).
- Those that have strong abstraction (allow programming paradigms independent of the machine details, such as complex variables, functions and looping that do not translate directly into machine code) are called **high-level languages**.
- Some languages, like C, are in between– allowing both low level access and high level data structures.

Machine Language



(Ruth Gordon & Ester Gerston programming the ENIAC, UPenn)

Machine Language

```
1 FOX 12:01a 23- 1
A 002000 C2 30 REP #$30
A 002002 18 CLC
A 002003 F8 SED
A 002004 A9 34 12 LDA #$1234
A 002007 69 21 43 ADC #$4321
A 00200A 8F 03 7F 01 STA $017F03
A 00200E D8 CLD
A 00200F E2 30 SEP #$30
A 002011 00 BRK
A 2012

r
PB PC NUmxDI2C .A .X .Y SP DP DB
; 00 E012 00110000 0000 0000 0002 CFFF 0000 00
g 2000

BREAK

PB PC NUmxDI2C .A .X .Y SP DP DB
; 00 2013 00110000 5555 0000 0002 CFFF 0000 00
m 7f03 7f03
>007F03 55 55 00 00 00 00 00 00 00 00 00 00 00 00 00:UU.....
█
```

(wiki)

Machine Language

- We will be writing programs in a simplified machine language, WeMIPS.

[illegible]

(wiki)

Machine Language



```
002000 c2 30      REP #430
002002 10          CLC
002003 F0          SED
002004 40 34 12     L3H 001234
002007 60 21 43     R0C 004321
00200A 0F 03 7F 01  STA 0017F03
00200C 00          CLJ
00200F E2 30      SEP #430
002011 00          BRX
002012

PC PC Mno32C A X Y SP BP BB
: 00 2012 00110000 0000 0000 0002 C7FF 0000 00
$ 2000

BREAK

PC PC Mno32C A X Y SP BP BB
: 00 2013 00110000 5555 0000 0002 C7FF 0000 00
n 1103 7403
007F03 55 55 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

(wiki)

- We will be writing programs in a simplified machine language, WeMIPS.
- It is based on a reduced instruction set computer (RISC) design, originally developed by the MIPS Computer Systems.

Machine Language



```
002000 c2 30      REP #030
002002 10          CLC
002003 f0          SED
002004 40 34 12    L3H #01234
002007 60 21 43    R0C #04321
002008 0f 03 7f 01 ST0 #017f03
00200c 00          CLJ
00200f e2 30      SEP #030
002011 00          BRX
002012

P0 PC 00000000 A X Y SP BP BB
: 00 0012 00110000 0000 0000 0002 c7ff 0000 00
$ 2000

BREAK

P0 PC 00000000 A X Y SP BP BB
: 00 2013 00110000 5555 0000 0002 c7ff 0000 00
n 1103 7f03
007f03 55 55 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

(wiki)

- We will be writing programs in a simplified machine language, WeMIPS.
- It is based on a reduced instruction set computer (RISC) design, originally developed by the MIPS Computer Systems.
- Due to its small set of commands, processors can be designed to run those commands very efficiently.

Machine Language



```
002000 c2 30 REP #430
002002 10 CLC
002003 f0 SED
002004 40 34 12 LSH #1224
002007 60 21 43 RLC #4321
00200a 0f 03 7f 01 STW #017f03
00200c 00 CLD
00200f e2 30 SEP #430
002011 00 BRX
002012

PC PC Hex32C A X Y SP BP BB
: 00 f012 00110000 0000 0000 0002 cfff 0000 00
$ 2000
BREAK
PC PC Hex32C A X Y SP BP BB
: 00 2013 00110000 6555 0000 0002 cfff 0000 00
n 1103 7f03
007f03 55 55 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

(wiki)

- We will be writing programs in a simplified machine language, WeMIPS.
- It is based on a reduced instruction set computer (RISC) design, originally developed by the MIPS Computer Systems.
- Due to its small set of commands, processors can be designed to run those commands very efficiently.
- More in future architecture classes....

“Hello World!” in Simplified Machine Language

Line: 3 Go!

Show/Hide Demos

[User Guide](#) | [Unit Tests](#) | [Docs](#)

Addition Doubler

Stav

Looper

Stack Test

Hello World

Code Gen Save String

Interactive

Binary2 Decimal

Decimal2 Binary

Debug

```
1 # Store 'Hello world!' at the top of the stack
2 ADDI $sp, $sp, -13
3 ADDI $t0, $zero, 72 # H
4 SB $t0, 0($sp)
5 ADDI $t0, $zero, 101 # e
6 SB $t0, 1($sp)
7 ADDI $t0, $zero, 108 # l
8 SB $t0, 2($sp)
9 ADDI $t0, $zero, 108 # l
10 SB $t0, 3($sp)
11 ADDI $t0, $zero, 111 # o
12 SB $t0, 4($sp)
13 ADDI $t0, $zero, 32 # (space)
14 SB $t0, 5($sp)
15 ADDI $t0, $zero, 119 # w
16 SB $t0, 6($sp)
17 ADDI $t0, $zero, 111 # o
18 SB $t0, 7($sp)
19 ADDI $t0, $zero, 114 # r
20 SB $t0, 8($sp)
21 ADDI $t0, $zero, 108 # l
22 SB $t0, 9($sp)
23 ADDI $t0, $zero, 100 # d
24 SB $t0, 10($sp)
25 ADDI $t0, $zero, 33 # !
26 SB $t0, 11($sp)
27 ADDI $t0, $zero, 0 # (null)
28 SB $t0, 12($sp)
29
30 ADDI $v0, $zero, 4 # 4 is for print string
31 ADDI $a0, $sp, 0
32 syscall # print to the log
```

Step Run ☒ Enable auto switching

S T A V Stack Log

s0:	10
s1:	9
s2:	9
s3:	22
s4:	696
s5:	976
s6:	927
s7:	418

(WeMIPS)

WeMIPS

View: 3
out
Show/Hide Demos
User Guide | Unit Tests | Docs

Addition Doubler
Shw
Looper
Stack Test
Hello World

Code Gen Save String
Interactive
Binary2 Decimal
Decimal2 Binary

Debug
▶

```

1 # Store "Hello world" at the top of the stack
2 ASCII Strcpy strcpy
3 ASCII Strcpy, memcpy, 72 # W
4 CD Strcpy, 61(Sp)
5 ASCII Strcpy, memcpy, 101 # w
6 CD Strcpy, 11(Sp)
7 ASCII Strcpy, memcpy, 108 # l
8 CD Strcpy, 2(Sp)
9 ASCII Strcpy, memcpy, 108 # l
10 CD Strcpy, 3(Sp)
11 ASCII Strcpy, memcpy, 111 # d
12 CD Strcpy, 4(Sp)
13 ASCII Strcpy, memcpy, 32 # (space)
14 CD Strcpy, 5(Sp)
15 ASCII Strcpy, memcpy, 119 # w
16 CD Strcpy, 6(Sp)
17 ASCII Strcpy, memcpy, 111 # o
18 CD Strcpy, 7(Sp)
19 ASCII Strcpy, memcpy, 114 # r
20 CD Strcpy, 8(Sp)
21 ASCII Strcpy, memcpy, 108 # l
22 CD Strcpy, 9(Sp)
23 ASCII Strcpy, memcpy, 105 # d
24 CD Strcpy, 10(Sp)
25 ASCII Strcpy, memcpy, 33 # !
26 CD Strcpy, 11(Sp)
27 ASCII Strcpy, memcpy, 0 # (null)
28 CD Strcpy, 12(Sp)
29
30 ASCII Strcpy, memcpy, 4 # 4 in for print string
31 ASCII Strcpy, memcpy, 0
32 printf
          
```

Step	Run	Enable auto switching
S	T	A V Stack Log
		10
str:		9
str:		9
str:		22
str:		350
str:		976
str:		927
str:		418

(Demo with WeMIPS)

MIPS Commands

```
1 # Choose "hello world" as the top of the stack
2 ADDI $0, $zero, 12345
3 SW $0, 0($0)
4 ADDI $0, $zero, 191 # 1
5 SW $0, 100($0)
6 ADDI $0, $zero, 100 # 2
7 SW $0, 200($0)
8 ADDI $0, $zero, 111 # 3
9 SW $0, 300($0)
10 ADDI $0, $zero, 12 # (space)
11 SW $0, 400($0)
12 ADDI $0, $zero, 119 # 1
13 SW $0, 500($0)
14 ADDI $0, $zero, 111 # 1
15 SW $0, 600($0)
16 ADDI $0, $zero, 114 # 1
17 SW $0, 700($0)
18 ADDI $0, $zero, 100 # 1
19 SW $0, 800($0)
20 ADDI $0, $zero, 100 # 2
21 SW $0, 900($0)
22 ADDI $0, $zero, 33 # 1
23 SW $0, 1000($0)
24 ADDI $0, $zero, 0 # (null)
25 SW $0, 1100($0)
26 ADDI $0, $zero, 4 # 4 is for print string
27 ADDI $0, $zero, 0 # print to the log
28 syscall
```

- **Registers:** locations for storing information that can be quickly accessed.

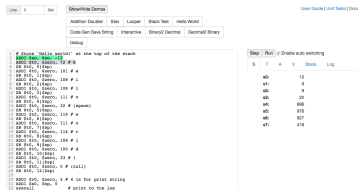
MIPS Commands

MIPS Commands

[illegible]

- **Registers:** locations for storing information that can be quickly accessed. Names start with '\$': \$s0, \$s1, \$t0, \$t1,...
- **R Instructions:** Commands that use data in the registers:

MIPS Commands



```
1 # Please "hello world" as the sign of the success
2 addi $v0, $zero, 12 # 12
3 li $t0, 1000000
4 add $s0, $zero, 100 # 100
5 li $t1, 1000000
6 add $s1, $zero, 100 # 1
7 li $t2, 1000000
8 add $s2, $zero, 100 # 1
9 li $t3, 1000000
10 add $s3, $zero, 100 # 1
11 li $t4, 1000000
12 add $s4, $zero, 12 # (space)
13 li $t5, 1000000
14 add $s5, $zero, 100 # 1
15 li $t6, 1000000
16 add $s6, $zero, 100 # 1
17 li $t7, 1000000
18 add $s7, $zero, 100 # 1
19 li $t8, 1000000
20 add $s8, $zero, 100 # 1
21 li $t9, 1000000
22 add $s9, $zero, 100 # 1
23 li $t10, 1000000
24 add $s10, $zero, 100 # 1
25 li $t11, 1000000
26 add $s11, $zero, 100 # 1
27 li $t12, 1000000
28 add $s12, $zero, 100 # 1
29 li $t13, 1000000
30 add $s13, $zero, 100 # 1
31 li $t14, 1000000
32 add $s14, $zero, 100 # 1
33 li $t15, 1000000
34 add $s15, $zero, 100 # 1
35 li $t16, 1000000
36 add $s16, $zero, 100 # 1
37 li $t17, 1000000
38 add $s17, $zero, 100 # 1
39 li $t18, 1000000
40 add $s18, $zero, 100 # 1
41 li $t19, 1000000
42 add $s19, $zero, 100 # 1
43 li $t20, 1000000
44 add $s20, $zero, 100 # 1
45 li $t21, 1000000
46 add $s21, $zero, 100 # 1
47 li $t22, 1000000
48 add $s22, $zero, 100 # 1
49 li $t23, 1000000
50 add $s23, $zero, 100 # 1
51 li $t24, 1000000
52 add $s24, $zero, 100 # 1
53 li $t25, 1000000
54 add $s25, $zero, 100 # 1
55 li $t26, 1000000
56 add $s26, $zero, 100 # 1
57 li $t27, 1000000
58 add $s27, $zero, 100 # 1
59 li $t28, 1000000
60 add $s28, $zero, 100 # 1
61 li $t29, 1000000
62 add $s29, $zero, 100 # 1
63 li $t30, 1000000
64 add $s30, $zero, 100 # 1
65 li $t31, 1000000
66 add $s31, $zero, 100 # 1
67 syscall
68 # print to the log
```

- **Registers:** locations for storing information that can be quickly accessed. Names start with '\$': \$s0, \$s1, \$t0, \$t1,...
- **R Instructions:** Commands that use data in the registers:
add \$s1, \$s2, \$s3 (Basic form: OP rd, rs, rt)
- **I Instructions:** instructions that also use intermediate values.

MIPS Commands

[illegible]

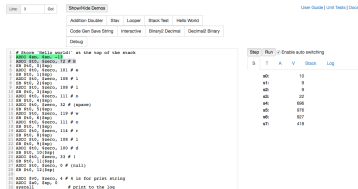
- **Registers:** locations for storing information that can be quickly accessed. Names start with '\$': \$s0, \$s1, \$t0, \$t1,...
- **R Instructions:** Commands that use data in the registers:
add \$s1, \$s2, \$s3 (Basic form: OP rd, rs, rt)
- **I Instructions:** instructions that also use intermediate values.
addi \$s1, \$s2, 100

MIPS Commands

[illegible]

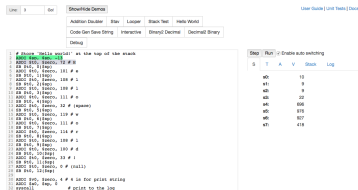
- **Registers:** locations for storing information that can be quickly accessed. Names start with '\$': \$s0, \$s1, \$t0, \$t1,...
- **R Instructions:** Commands that use data in the registers:
add \$s1, \$s2, \$s3 (Basic form: OP rd, rs, rt)
- **I Instructions:** instructions that also use intermediate values.
addi \$s1, \$s2, 100 (Basic form: OP rd, rs, imm)
- **J Instructions:** instructions that jump to another memory location.

MIPS Commands



- **Registers:** locations for storing information that can be quickly accessed. Names start with '\$': \$s0, \$s1, \$t0, \$t1,...
- **R Instructions:** Commands that use data in the registers:
add \$s1, \$s2, \$s3 (Basic form: OP rd, rs, rt)
- **I Instructions:** instructions that also use intermediate values.
addi \$s1, \$s2, 100 (Basic form: OP rd, rs, imm)
- **J Instructions:** instructions that jump to another memory location.
j done

MIPS Commands



```
1 # Please "Hello world!" as the top of the main
2 addi $v0, $zero, 12 # 12
3 li $t0, 1000000
4 addi $a0, $zero, 100 # 100
5 li $t1, 1000000
6 addi $a1, $zero, 100 # 100
7 li $t2, 1000000
8 addi $a2, $zero, 100 # 100
9 li $t3, 1000000
10 addi $a3, $zero, 100 # 100
11 li $t4, 1000000
12 addi $a4, $zero, 100 # 100
13 li $t5, 1000000
14 addi $a5, $zero, 100 # 100
15 li $t6, 1000000
16 addi $a6, $zero, 100 # 100
17 li $t7, 1000000
18 addi $a7, $zero, 100 # 100
19 li $t8, 1000000
20 addi $a8, $zero, 100 # 100
21 li $t9, 1000000
22 addi $a9, $zero, 100 # 100
23 li $t10, 1000000
24 addi $a10, $zero, 100 # 100
25 li $t11, 1000000
26 addi $a11, $zero, 100 # 100
27 li $t12, 1000000
28 addi $a12, $zero, 100 # 100
29 li $t13, 1000000
30 addi $a13, $zero, 100 # 100
31 li $t14, 1000000
32 addi $a14, $zero, 100 # 100
33 li $t15, 1000000
34 addi $a15, $zero, 100 # 100
35 li $t16, 1000000
36 addi $a16, $zero, 100 # 100
37 li $t17, 1000000
38 addi $a17, $zero, 100 # 100
39 li $t18, 1000000
40 addi $a18, $zero, 100 # 100
41 li $t19, 1000000
42 addi $a19, $zero, 100 # 100
43 li $t20, 1000000
44 addi $a20, $zero, 100 # 100
45 li $t21, 1000000
46 addi $a21, $zero, 100 # 100
47 li $t22, 1000000
48 addi $a22, $zero, 100 # 100
49 li $t23, 1000000
50 addi $a23, $zero, 100 # 100
51 li $t24, 1000000
52 addi $a24, $zero, 100 # 100
53 li $t25, 1000000
54 addi $a25, $zero, 100 # 100
55 li $t26, 1000000
56 addi $a26, $zero, 100 # 100
57 li $t27, 1000000
58 addi $a27, $zero, 100 # 100
59 li $t28, 1000000
60 addi $a28, $zero, 100 # 100
61 li $t29, 1000000
62 addi $a29, $zero, 100 # 100
63 li $t30, 1000000
64 addi $a30, $zero, 100 # 100
65 li $t31, 1000000
66 addi $a31, $zero, 100 # 100
67 syscall
68 # print to the log
```

\$	T	A	V	Stack	Log
\$0			10		
\$1			9		
\$2			9		
\$3			22		
\$4			906		
\$5			917		
\$6			418		

- **Registers:** locations for storing information that can be quickly accessed. Names start with '\$': \$s0, \$s1, \$t0, \$t1,...
- **R Instructions:** Commands that use data in the registers:
add \$s1, \$s2, \$s3 (Basic form: OP rd, rs, rt)
- **I Instructions:** instructions that also use intermediate values.
addi \$s1, \$s2, 100 (Basic form: OP rd, rs, imm)
- **J Instructions:** instructions that jump to another memory location.
j done (Basic form: OP label)

Try it yourself:

Line: 3 Go!

Show/Hide Demos

[User Guide](#) | [Unit Tests](#) | [Docs](#)

Addition Doubler

Stav

Looper

Stack Test

Hello World

Code Gen Save String

Interactive

Binary2 Decimal

Decimal2 Binary

Debug

```
1 # Store 'Hello world!' at the top of the stack
2 ADDI $sp, $sp, -13
3 ADDI $t0, $zero, 72 # H
4 SB $t0, 0($sp)
5 ADDI $t0, $zero, 101 # e
6 SB $t0, 1($sp)
7 ADDI $t0, $zero, 108 # l
8 SB $t0, 2($sp)
9 ADDI $t0, $zero, 108 # l
10 SB $t0, 3($sp)
11 ADDI $t0, $zero, 111 # o
12 SB $t0, 4($sp)
13 ADDI $t0, $zero, 32 # (space)
14 SB $t0, 5($sp)
15 ADDI $t0, $zero, 119 # w
16 SB $t0, 6($sp)
17 ADDI $t0, $zero, 111 # o
18 SB $t0, 7($sp)
19 ADDI $t0, $zero, 114 # r
20 SB $t0, 8($sp)
21 ADDI $t0, $zero, 108 # l
22 SB $t0, 9($sp)
23 ADDI $t0, $zero, 100 # d
24 SB $t0, 10($sp)
25 ADDI $t0, $zero, 33 # !
26 SB $t0, 11($sp)
27 ADDI $t0, $zero, 0 # (null)
28 SB $t0, 12($sp)
29
30 ADDI $v0, $zero, 4 # 4 is for print string
31 ADDI $a0, $sp, 0
32 syscall # print to the log
```

Step Run ☒ Enable auto switching

S	T	A	V	Stack	Log
				s0:	10
				s1:	9
				s2:	9
				s3:	22
				s4:	696
				s5:	976
				s6:	927
				s7:	418

Write a program that prints out the alphabet: a b c d ... x y z

WeMIPS

Line 3 col 12

Show/Hide Demos

User Guide | Unit Tests | Docs

Addition Doubler | Stop | Looper | Stack Test | Hello World

Code Gen Save String | Interactive | Binary2 Decimal | Decimal2 Binary

Debug

```
1 # Store "hello world!" at the top of the stack
2 ADDUI $a0, $zero, 0
3 ADDUI $a0, $zero, 32 # 4
4 ROR $t0, $t0, 1
5 ADDUI $t0, $zero, 101 # e
6 DD $t0, 1($t0)
7 ADDUI $t0, $zero, 108 # l
8 DD $t0, 6($t0)
9 ADDUI $t0, $zero, 108 # l
10 DD $t0, 6($t0)
11 ADDUI $t0, $zero, 111 # o
12 DD $t0, 6($t0)
13 ADDUI $t0, $zero, 32 # (space)
14 DD $t0, 6($t0)
15 ADDUI $t0, $zero, 119 # w
16 DD $t0, 6($t0)
17 ADDUI $t0, $zero, 114 # u
18 DD $t0, 7($t0)
19 ADDUI $t0, $zero, 114 # u
20 DD $t0, 6($t0)
21 ADDUI $t0, $zero, 108 # l
22 DD $t0, 6($t0)
23 ADDUI $t0, $zero, 108 # l
24 DD $t0, 10($t0)
25 ADDUI $t0, $zero, 33 # !
26 DD $t0, 11($t0)
27 ADDUI $t0, $zero, 0 # (null)
28 DD $t0, 12($t0)
29
30 ADDUI $v0, $zero, 6 # 4 in for print string
31 ADDUI $a0, $a0, 0
32 syscall # print to the log
```

Step | Run | Enable auto-switching

S	T	A	V	Stack	Log
				a0:	10
				t0:	9
				a0:	9
				a0:	22
				a0:	695
				a0:	970
				a0:	927
				a0:	418

(Demo with WeMIPS)

Today's Topics



- Machine Language
- **Loops & Labels in MIPS**
- C++: Basic Format & Variables
- I/O and Definite Loops in C++
- Challenge Questions

Loops & Jumps in Machine Language

- Instead of built-in looping structures like `for` and `while`, you create your own loops by “jumping” to the location in the program.



Loops & Jumps in Machine Language

- Instead of built-in looping structures like `for` and `while`, you create your own loops by “jumping” to the location in the program.
- Can indicate locations by writing **labels** at the beginning of a line.



Loops & Jumps in Machine Language

- Instead of built-in looping structures like `for` and `while`, you create your own loops by “jumping” to the location in the program.
- Can indicate locations by writing **labels** at the beginning of a line.
- Then give a command to jump to that location.
- Different kinds of jumps:



Loops & Jumps in Machine Language

- Instead of built-in looping structures like `for` and `while`, you create your own loops by “jumping” to the location in the program.
- Can indicate locations by writing **labels** at the beginning of a line.
- Then give a command to jump to that location.
- Different kinds of jumps:
 - ▶ **Unconditional:** `j Done` will jump to the address with label `Done`.



Loops & Jumps in Machine Language

- Instead of built-in looping structures like `for` and `while`, you create your own loops by “jumping” to the location in the program.
- Can indicate locations by writing **labels** at the beginning of a line.
- Then give a command to jump to that location.
- Different kinds of jumps:
 - ▶ **Unconditional:** `j Done` will jump to the address with label `Done`.
 - ▶ **Branch if Equal:** `beq $s0 $s1 DoAgain` will jump to the address with label `DoAgain` if the registers `$s0` and `$s1` contain the same value.



Loops & Jumps in Machine Language

- Instead of built-in looping structures like `for` and `while`, you create your own loops by “jumping” to the location in the program.
- Can indicate locations by writing **labels** at the beginning of a line.
- Then give a command to jump to that location.
- Different kinds of jumps:
 - ▶ **Unconditional:** `j Done` will jump to the address with label `Done`.
 - ▶ **Branch if Equal:** `beq $s0 $s1 DoAgain` will jump to the address with label `DoAgain` if the registers `$s0` and `$s1` contain the same value.
 - ▶ See reading for more variations.



MIPS: Print Alphabet Example

```
ADDI $sp, $sp, -27    #setup stack, 26 letters + 1 null
ADDI $t0, $zero, 97   #save ASCII of 'a' to t0
ADDI $s2, $zero, 26   #set s2 to 26, used to check loop
```

```
SETUP:
SB $t0, 0($sp)        #save contents of t0 to stack
ADDI $sp, $sp, 1       #increment the stack
ADDI $s2, $s2, -1      #subtract 1 from s2
ADDI $t0, $t0, 1       #increment the letter
BEQ $s2, $zero, DONE
J SETUP
```

```
DONE:
ADDI $t0, $zero, 0     #set null
SB $t0, 0($sp)
ADDI $sp, $sp, -26
ADDI $v0, $zero, 4     #v0 is 4 for print syscall
ADDI $a0, $sp, 0       #set a0 to stack pointer
syscall
```

Quiz points

```
addi $sp, $sp, -6
addi $t0, $zero, 90
addi $s2, $zero, 5
SETUP: sb $t0, 0($sp)
addi $sp, $sp, 1
addi $s2, $s2, -1
beq $s2, $zero, DONE
j SETUP
DONE: addi $t0, $zero, 0
sb $t0, 0($sp)
addi $sp, $sp, -6
addi $v0, $zero, 4
addi $a0, $sp, 0
syscall
```

- Run in the WeMIPS emulator. What is the output?
- What's the fewest changes to output: YYYYYY?
- What's the fewest changes to output: ZYXWV?

Today's Topics



- Machine Language
- Loops & Labels in MIPS
- **C++: Basic Format & Variables**
- I/O and Definite Loops in C++
- Challenge Questions

Challenge:

- Using what you know from Python, predict what the C++ code will do:

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello |" << year << "!!\n\n";
11    return 0;
12 }
```

onlinedb demo

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello !" << year << "!\n\n";
11    return 0;
12 }
```

(Demo with onlinedb)

Introduction to C++

- C++ is a popular programming language that extends C.

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello ! << year << "!!\n\n";
11    return 0;
12 }
```


Introduction to C++

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello ! << year << "!!\n\n";
11    return 0;
12 }
```

- C++ is a popular programming language that extends C.
- Fast, efficient, and powerful.

Introduction to C++

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello ! << year << "!!\n\n";
11    return 0;
12 }
```

- C++ is a popular programming language that extends C.
- Fast, efficient, and powerful.
- Used for systems programming (and future courses!).

Introduction to C++

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello ! << year << "!!\n\n";
11    return 0;
12 }
```

- C++ is a popular programming language that extends C.
- Fast, efficient, and powerful.
- Used for systems programming (and future courses!).
- Today, we'll introduce the basic structure and simple input/output (I/O) in C/C++.

Introduction to C++

- Programs are organized in functions.

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello " << year << "!!\n\n";
11    return 0;
12 }
```

Introduction to C++

- Programs are organized in functions.
- Variables must be **declared**:

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello " << year << "!!\n\n";
11    return 0;
12 }
```

Introduction to C++

- Programs are organized in functions.
- Variables must be **declared**:
`int num;`

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello " << year << "!!\n\n";
11    return 0;
12 }
```

Introduction to C++

- Programs are organized in functions.
- Variables must be **declared**:
`int num;`
- Many types available:
`int, float, char, ...`

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello " << year << "!!\n\n";
11    return 0;
12 }
```

Introduction to C++

- Programs are organized in functions.
- Variables must be **declared**:
`int num;`
- Many types available:
`int, float, char, ...`
- Semicolons separate commands:

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello " << year << "!!\n\n";
11    return 0;
12 }
```


Introduction to C++

- Programs are organized in functions.
- Variables must be **declared**:
`int num;`
- Many types available:
`int, float, char, ...`
- Semicolons separate commands:
`num = 5; more = 2*num;`

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello " << year << "!!\n\n";
11    return 0;
12 }
```

Introduction to C++

- Programs are organized in functions.
- Variables must be **declared**:
`int num;`
- Many types available:
`int, float, char, ...`
- Semicolons separate commands:
`num = 5; more = 2*num;`
- To print, we'll use `cout <<`:

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello " << year << "!!\n\n";
11    return 0;
12 }
```

Introduction to C++

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello ! << year << "!!\n\n";
11    return 0;
12 }
```

- Programs are organized in functions.
- Variables must be **declared**:
`int num;`
- Many types available:
`int, float, char, ...`
- Semicolons separate commands:
`num = 5; more = 2*num;`
- To print, we'll use `cout <<`:
`cout << "Hello!!";`

Introduction to C++

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello ! << year << "!!\n\n";
11    return 0;
12 }
```

- Programs are organized in functions.
- Variables must be **declared**:
`int num;`
- Many types available:
`int, float, char, ...`
- Semicolons separate commands:
`num = 5; more = 2*num;`
- To print, we'll use `cout <<`:
`cout << "Hello!!";`
- To get input, we'll use `cin >>`:

Introduction to C++

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello ! << year << "!!\n\n";
11    return 0;
12 }
```

- Programs are organized in functions.
- Variables must be **declared**:
`int num;`
- Many types available:
`int, float, char, ...`
- Semicolons separate commands:
`num = 5; more = 2*num;`
- To print, we'll use `cout <<`:
`cout << "Hello!!";`
- To get input, we'll use `cin >>`:
`cin >> num;`

Introduction to C++

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello ! << year << "!!\n\n";
11    return 0;
12 }
```

- Programs are organized in functions.
- Variables must be **declared**:
`int num;`
- Many types available:
`int, float, char, ...`
- Semicolons separate commands:
`num = 5; more = 2*num;`
- To print, we'll use `cout <<`:
`cout << "Hello!!";`
- To get input, we'll use `cin >>`:
`cin >> num;`
- To use those I/O functions, we put at the top of the program:

Introduction to C++

```
1 //Another C++ program, demonstrating variables
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int year;
8     cout << "Enter a number: ";
9     cin >> year;
10    cout << "Hello ! << year << "!!\n\n";
11    return 0;
12 }
```

- Programs are organized in functions.
- Variables must be **declared**:
`int num;`
- Many types available:
`int, float, char, ...`
- Semicolons separate commands:
`num = 5; more = 2*num;`
- To print, we'll use `cout <<`:
`cout << "Hello!!";`
- To get input, we'll use `cin >>`:
`cin >> num;`
- To use those I/O functions, we put at the top of the program:
`#include <iostream>`
`using namespace std;`

Challenge:

Predict what the following pieces of code will do:

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```


C++ Demo

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

(Demo with onlinegdb)

Python Tutor

Convert the C++ code to a **Python** program:

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

(Write from scratch in pythonTutor.)

Today's Topics



- Machine Language
- Loops & Labels in MIPS
- C++: Basic Format & Variables
- **I/O and Definite Loops in C++**
- Challenge Questions

Challenge:

Predict what the following pieces of code will do:

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!" << endl;

    return 0;
}
```

C++ Demo

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!" << endl;

    return 0;
}
```

(Demo with onlinedb)

Definite loops

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!" << endl;

    return 0;
}
```

General format:

```
for ( initialization ; test ; updateAction )
{
    command1;
    command2;
    command3;
    ...
}
```

Challenge:

Predict what the following pieces of code will do:

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j,size;
    cout << "Enter size: ";
    cin >> size;
    for (i = 0; i < size; i++)
    {
        for (j = 0; j < size; j++)
        {
            cout << "*";
            cout << endl;
        }
        cout << "\n\n";
        for (i = size; i > 0; i--)
        {
            for (j = 0; j < i; j++)
            {
                cout << "*";
                cout << endl;
            }
        }
        return 0;
    }
}
```

C++ Demo

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j,size;
    cout << "Enter size: ";
    cin >> size;
    for (i = 0; i < size; i++)
    {
        for (j = 0; j < size; j++)
        {
            cout << "*";
            cout << endl;
        }
        cout << "\n\n";
        for (i = size; i > 0; i--)
        {
            for (j = 0; j < i; j++)
            {
                cout << "*";
                cout << endl;
            }
        }
        return 0;
    }
}
```

(Demo with onlinegdb)

Quiz Point:

Translate the C++ program into Python:

```
#include <iostream>
using namespace std;

int main ()
{
    int population = 100;
    cout << "Year\tPopulation\n";
    for (int year=0; year<100; year=year+5)
    {
        cout<<year<<"\t"<<population<<"\n";
        population=population*2;
    }
    return 0;
}
```

Challenge:

Predict what the following pieces of code will do:

```
//Demonstrates conditionals
#include <iostream>
using namespace std;

int main ()
{
    int yearBorn;
    cout << "Enter year born: ";
    cin >> yearBorn;
    if (yearBorn < 1946)
    {
        cout << "Greatest Generation";
    }
    else if (yearBorn <= 1964)
    {
        cout << "Baby Boomer";
    }
    else if (yearBorn <= 1984)
    {
        cout << "Generation X";
    }
    else if (yearBorn <= 2004)
    {
        cout << "Millennial";
    }
    else
    {
        cout << "TBD";
    }
}
```

return 0:

CSci 127 (Hunter)

```
using namespace std;
```

```
int main ()
{
```

```
    string conditions = "blowing snow";
```

```
    int winds = 100;
```

```
    float visibility = 0.2;
```

```
    if ( ( (winds > 35) && (visibility < 0.25) )
        ( (conditions == "blowing snow") ||
          (conditions == "heavy snow") ) )
        cout << "Blizzard!\n";
```

```
    string origin = "South Pacific";
```

```
    if (winds > 74)
```

```
        cout << "Major storm, called a ";
```

```
    if ((origin == "Indian Ocean")
```

```
        || (origin == "South Pacific"))
```

```
        cout << "cyclone.\n";
```

```
    else if (origin == "North Pacific")
```

```
        cout << "typhoon.\n";
```

```
    else
```

```
        cout << "hurricane.\n";
```

Logical Operators in C++

Very similar, just different names: `&&`, `||`, and `!`:

Logical Operators in C++

Very similar, just different names: `&&`, `||`, and `!`:

and (&&)

in1		in2	<i>returns:</i>
False	<code>&&</code>	False	False
False	<code>&&</code>	True	False
True	<code>&&</code>	False	False
True	<code>&&</code>	True	True

Logical Operators in C++

Very similar, just different names: `&&`, `||`, and `!`:

and (`&&`)

in1		in2	<i>returns:</i>
False	<code>&&</code>	False	False
False	<code>&&</code>	True	False
True	<code>&&</code>	False	False
True	<code>&&</code>	True	True

or (`||`)

in1		in2	<i>returns:</i>
False	<code> </code>	False	False
False	<code> </code>	True	True
True	<code> </code>	False	True
True	<code> </code>	True	True

Logical Operators in C++

Very similar, just different names: `&&`, `||`, and `!`:

and (`&&`)

in1		in2	returns:
False	<code>&&</code>	False	False
False	<code>&&</code>	True	False
True	<code>&&</code>	False	False
True	<code>&&</code>	True	True

or (`||`)

in1		in2	returns:
False	<code> </code>	False	False
False	<code> </code>	True	True
True	<code> </code>	False	True
True	<code> </code>	True	True

not (`!`)

	in1	returns:
<code>!</code>	False	True
<code>!</code>	True	False

Quiz Point

- Write a C++ program that will ask for the time in 24 hour format and, prints out “Good Morning” if it is before noon, knowing it is morning before noon, “Good Evening” if it after 6pm, and otherwise print “Good Afternoon.”

Challenge: Predict what code will do:

```
//While Growth example
#include <iostream>
using namespace std;

int main ()
{
    int population = 100;
    int year = 0;
    cout << "Year\tPopulation\n";
    while (population < 1000)
    {
        cout << year << "\t" << population << "\n";
        population = population * 2;
        year++;
    }
    return 0;
}
```

OnlineGDB Demo

Indefinite Loops: while

```
///While Growth Example
#include <iostream>
using namespace std;

int main ()
{
    int population = 100;
    int year = 0;
    cout << "Year\\tPopulation\\n";
    while(population < 1000)
    {
        cout << year << "\\t\\t" << population << "\\n";
        population = population * 2;
        year++;
    }
    return 0;
}
```

General format:

```
while ( logical expression )
{
    command1;
    command2;
    command3;
    ...
}
```

Challenge: Predict what the code do

```
#include <iostream>
using namespace std;

int main ()
{
    int num;
    cout << "Enter an even number: ";
    cin >> num;
    while (num % 2 != 0)
    {
        cout << "\nThat's odd!\n";
        cout << "Enter an even number: ";
        cin >> num;
    }
    cout << "You entered: " << num << ".\n";
    return 0;
}
```

Demo: [OnlineGDB](#)

Today's Topics



- Machine Language
- Loops & Labels in MIPS
- C++: Basic Format & Variables
- I/O and Definite Loops in C++
- **Challenge Questions and Recap**

Challenge: Definite Loops in Python & C++

- *Rewrite this program in C++:*

```
for i in range(2017, 2000, -2):  
    print("Year is", i)
```

- *Rewrite this program in Python:*

```
#include <iostream>  
using namespace std;  
int main()  
{  
    for (int i = 1; i < 50; i++)  
    {  
        cout << i << endl;  
    }  
    return 0;  
}
```

Challenge: Definite Loops in Python & C++

- *Rewrite this program in C++:*

```
for i in range(2017, 2000, -2):  
    print("Year is", i)
```

Challenge: Definite Loops in Python & C++

- Rewrite this program in C++:

```
for i in range(2017, 2000, -2):  
    print("Year is", i)
```

- Answer:

```
#include <iostream>  
using namespace std;  
int main()  
{  
    for (int i = 2017; i > 2000; i=i-2)  
    {  
        cout << "Year is " << i << endl;  
    }  
    return 0;  
}
```

Challenge: Definite Loops in Python & C++

- *Rewrite this program in Python:*

```
#include <iostream>
using namespace std;
int main()
{
    for (int i = 1; i < 50; i++)
    {
        cout << i << endl;
    }
    return 0;
}
```

Challenge: Definite Loops in Python & C++

- *Rewrite this program in Python:*

```
#include <iostream>
using namespace std;
int main()
{
    for (int i = 1; i < 50; i++)
    {
        cout << i << endl;
    }
    return 0;
}
```

- Answer:

```
for i in range(1, 50):
    print(i)
```


Challenge: Conditionals in Python & C++

- *Write a C++ program that asks the user the number of times they plan to ride transit this week. Your program should then print if it is cheaper to buy single ride metro cards or 7-day unlimited card.
(The 7-day card is \$34.00, and the cost of single ride is \$2.90).*

Recap: Basic Form & I/O in C++

- Efficient for systems programming.

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

Recap: Basic Form & I/O in C++

- Efficient for systems programming.
- Programs are organized in functions.

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

Recap: Basic Form & I/O in C++

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables:

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

Recap: Basic Form & I/O in C++

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

Recap: Basic Form & I/O in C++

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

Recap: Basic Form & I/O in C++

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

Recap: Basic Form & I/O in C++

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`
- To print:

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```


Recap: Basic Form & I/O in C++

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`
- To print: `cout << "Hello!!";`

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

Recap: Basic Form & I/O in C++

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`
- To print: `cout << "Hello!!";`
- To get input:

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

Recap: Basic Form & I/O in C++

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`
- To print: `cout << "Hello!!";`
- To get input: `cin >> num;`

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

Recap: Basic Form & I/O in C++

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`
- To print: `cout << "Hello!!";`
- To get input: `cin >> num;`
- To use those I/O functions:

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

Recap: Basic Form & I/O in C++

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`
- To print: `cout << "Hello!!";`
- To get input: `cin >> num;`
- To use those I/O functions:
`#include <iostream>`
`using namespace std;`

Recap: Basic Form & I/O in C++

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`
- To print: `cout << "Hello!!";`
- To get input: `cin >> num;`
- To use those I/O functions:
`#include <iostream>`
`using namespace std;`
- Definite loops:

Recap: Basic Form & I/O in C++

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`
- To print: `cout << "Hello!!";`
- To get input: `cin >> num;`
- To use those I/O functions:
`#include <iostream>`
`using namespace std;`
- Definite loops:
`for (i = 0; i < 10; i++) {...}`

Recap: Basic Form & I/O in C++

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`
- To print: `cout << "Hello!!";`
- To get input: `cin >> num;`
- To use those I/O functions:
`#include <iostream>`
`using namespace std;`
- Definite loops:
`for (i = 0; i < 10; i++) {...}`
- Blocks of code uses '{' and '}'.

Recap: Basic Form & I/O in C++

```
//Another C++ program, demonstrating I/O & arithmetic
#include <iostream>
using namespace std;

int main ()
{
    float kg, lbs;
    cout << "Enter kg: ";
    cin >> kg;
    lbs = kg * 2.2;
    cout << endl << "Lbs: " << lbs << "\n\n";
    return 0;
}
```

- Efficient for systems programming.
- Programs are organized in functions.
- Must declare variables: `int num;`
- Many types available:
`int, float, char, ...`
- To print: `cout << "Hello!!";`
- To get input: `cin >> num;`
- To use those I/O functions:
`#include <iostream>`
`using namespace std;`
- Definite loops:
`for (i = 0; i < 10; i++) {...}`
- Blocks of code uses '{' and '}'.
- Commands generally end in ';'.

Recap: C++ Control Structures

- I/O:

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!!" << endl;

    return 0;
}
```

Recap: C++ Control Structures

- I/O: `cin >> ...;`

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!!" << endl;

    return 0;
}
```

Recap: C++ Control Structures

- I/O: `cin >> ...;` & `cout << ...;`

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!!" << endl;

    return 0;
}
```

Recap: C++ Control Structures

- I/O: `cin >> ...;` & `cout << ...;`
- Definite loops:

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!!" << endl;

    return 0;
}
```

Recap: C++ Control Structures

- I/O: `cin >> ...; & cout << ...;`
- Definite loops:
`for (i = 0; i < 10; i++)`
`{`
`...`
`}`

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!!" << endl;

    return 0;
}
```

Recap: C++ Control Structures

- I/O: `cin >> ...; & cout << ...;`
- Definite loops:
`for (i = 0; i < 10; i++)`
`{`

`...`
`}`
- Conditionals:

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!!" << endl;

    return 0;
}
```

Recap: C++ Control Structures

- I/O: `cin >> ...; & cout << ...;`

- Definite loops:

```
for (i = 0; i < 10; i++)  
{  
    ...  
}
```

- Conditionals:

```
if (logical expression)  
{  
    ...  
}  
else  
{  
    ...  
}
```

```
//Another C++ program; Demonstrates loops  
#include <iostream>  
using namespace std;  
  
int main ()  
{  
    int i,j;  
    for (i = 0; i < 4; i++)  
    {  
        cout << "The world turned upside down...\n";  
    }  
  
    for (j = 10; j > 0; j--)  
    {  
        cout << j << " ";  
    }  
    cout << "Blast off!!!" << endl;  
    return 0;  
}
```


Recap: C++ Control Structures

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!!" << endl;

    return 0;
}
```

- I/O: `cin >> ...; & cout << ...;`
- Definite loops:
`for (i = 0; i < 10; i++)`
{
 ...
}
- Conditionals:
`if (logical expression)`
{
 ...
}
`else`
{
 ...
}
- Indefinite loops:

Recap: C++ Control Structures

```
//Another C++ program; Demonstrates loops
#include <iostream>
using namespace std;

int main ()
{
    int i,j;
    for (i = 0; i < 4; i++)
    {
        cout << "The world turned upside down...\n";
    }

    for (j = 10; j > 0; j--)
    {
        cout << j << " ";
    }
    cout << "Blast off!!!" << endl;

    return 0;
}
```

- I/O: `cin >> ...; & cout << ...;`

- Definite loops:

```
for (i = 0; i < 10; i++)
{
    ...
}
```

- Conditionals:

```
if (logical expression)
{
    ...
}
else
{
    ...
}
```

- Indefinite loops:

```
while (logical expression)
{
    ...
}
```